Unified Engineering Workbench: Technical Report

Candidate ID: RA2311056010232

1. Design choices

The backend is C++17 and the front end is a browser page served from the same executable. Core logic (FEA, parsers, thread pool, batch, backprop, mock devices) lives in a static library, wb_core, with no HTTP or UI code. A thin server layer maps JSON requests onto it. Launch.exe starts the server on 127.0.0.1 only, on a free port, opens the default browser and exits when the page stops sending heartbeats, so no terminal, install or firewall prompt is needed. MSVC builds use the static runtime, and CI checks that Launch.exe imports no VC++ runtime DLL. Three.js and Chart.js are bundled locally, so the app works offline.

Two inputs are supported. An ASCII STL is parsed and drawn as a rotatable, pannable, zoomable surface (units are model units because STL has no unit field). A nodal CSV with the header node_id,x_m,y_m,z_m,temperature_C (metres, degrees C) is drawn as a colour-mapped point cloud with a legend; clicking a node shows its id, coordinates and temperature. Malformed files are rejected with a line-numbered message and never crash the server. The bundled examples are synthetic and labelled as such in data/README.md.

2. Thermal FEA verification

Two-node linear elements assemble K = (kA/h)[[1,-1],[-1,1]] and a consistent load qAh/2 per node. Dirichlet values are moved to the right-hand side and the constrained rows become identity rows. The error is measured at element midpoints from the FE interpolation, not only at nodes.

N	h [m]	Nodal error [C]	Midpoint error [C]	Order
10	0.1	2.8e-14	1.25	-
20	0.05	2.0e-13	0.3125	2.00
40	0.025	8.8e-13	0.0781	2.00
80	0.0125	1.2e-12	0.0195	2.00

The midpoint error equals (q/2k)h²/4 exactly, which a unit test checks. Debugging insight: nodal values are exact to round-off for a quadratic solution, so a nodes-only check would hide the discretisation error entirely; the h² behaviour only appears between nodes.

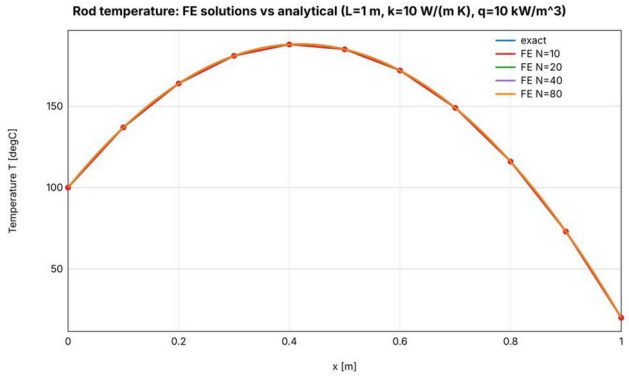


Fig. 1a: FE vs exact, N = 10 to 80.

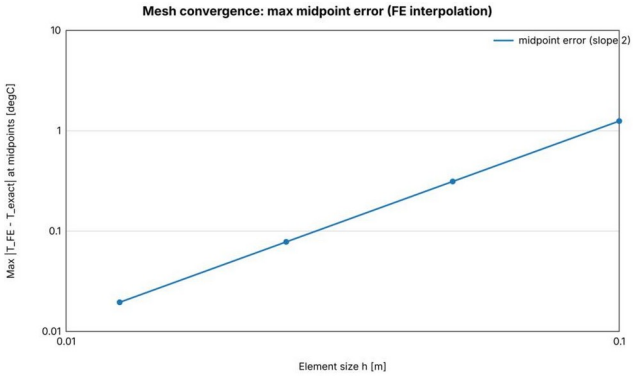


Fig. 1b: midpoint error vs h.

3. Performance results

Workload: 20,000 independent solves of 2,000 elements, seed 12345, results kept in input order. A bounded std::thread pool is created outside the timed

region. Protocol: one warm-up, five measured runs, median reported; only the solve is timed. Machine: Apple M4 (10 cores, 10 logical), 16 GiB, macOS, Apple Clang 17, -O3 -DNDEBUG, a passively cooled MacBook Air.

Workers	Run 1 [s]	Run 2 [s]	Speedup	Efficiency
1	0.2348	0.2453	1.00	1.00
2	0.1242	0.1234	1.89 / 1.99	0.95 / 0.99
4	0.0668	0.0668	3.51 / 3.67	0.88 / 0.92

Serial and parallel temperatures agree exactly (maximum difference 0, limit 1e-8 C). The parallel medians repeat within 1% between runs, while the serial median moved by 4% (individual serial runs ranged from 0.228 to 0.275 s), so most of the speedup difference between runs comes from the serial baseline. Efficiency falls at 4 workers because of thread start-up and queueing, memory bandwidth, and a short run (about 0.07 s) where small noise matters. Throttling is possible but was not measured. The Windows build was only tested in a virtual machine, so no Windows timings are claimed. Launch-to-ready time is reported separately in the app footer (Windows VM: server ready 6 ms and UI loaded 429 ms after process start, in a Windows 11 ARM virtual machine with 4 logical CPUs running the x64 build under emulation).

4. Embedded design

The design (docs/embedded_design.md) separates three voltage domains for the microcontroller, FPGA and JTAG adapter. The adapter drives TCK, TMS and TDI, the target drives TDO, and VTref is sensed, not supplied. A JTAG adapter is a debug interface: it runs no application and its telemetry describes the link. No shared connector, voltage or protocol is assumed. The C++ mock adapters demonstrate discovery, capabilities, connect, disconnect, telemetry and a recoverable timeout with retry; everything is labelled SIMULATED.

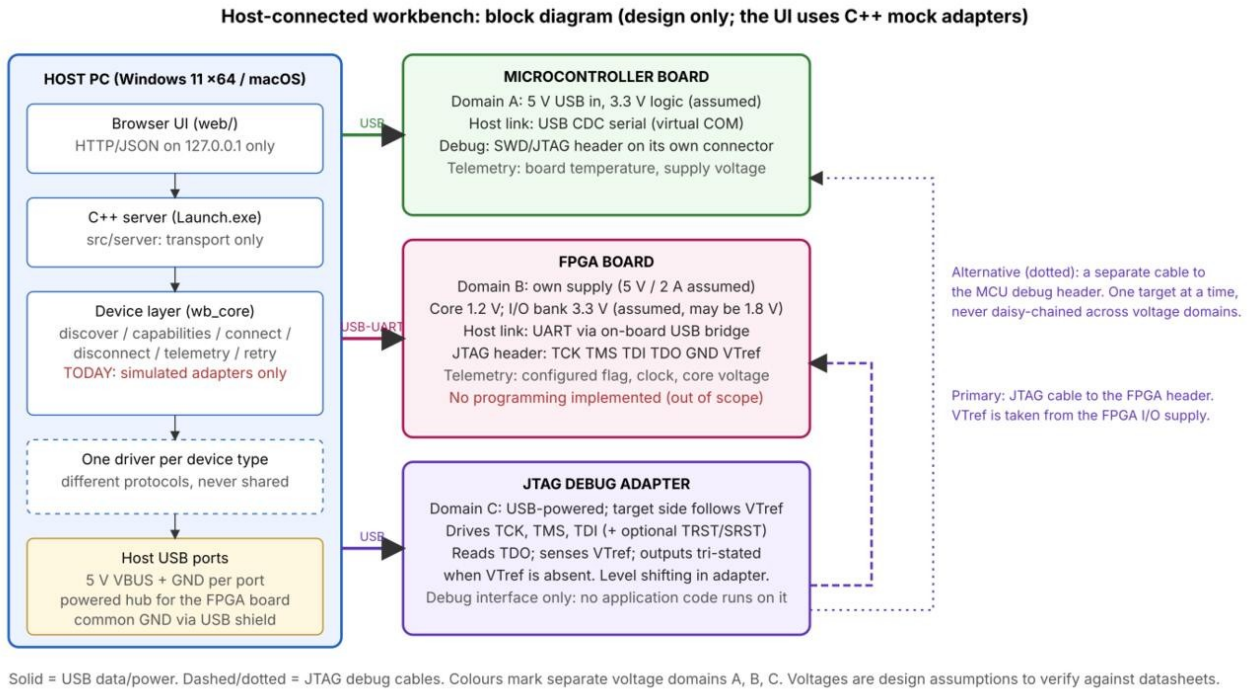


Fig. 2: block diagram (design only).

5. Backpropagation

With x=1, y=2, w=0.5, b=0, v=1, c=0 and learning rate 0.1: z=0.5, h=0.4621, y_hat=0.4621, J=1.1825. The gradients are dJ/dv=-0.7107, dJ/dc=-1.5379, dJ/dw=dJ/db=-1.2095, all negative, so every parameter increases and the prediction moves toward the target. After one simultaneous update J=0.6858. Central differences (epsilon 1e-6) agree with the analytic gradients to 1.1e-10 (limit 1e-5). Over 100 steps the loss falls monotonically to 1.7e-20 (Fig. 3); one sample can be fitted exactly, so this shows correct descent, not generalisation.

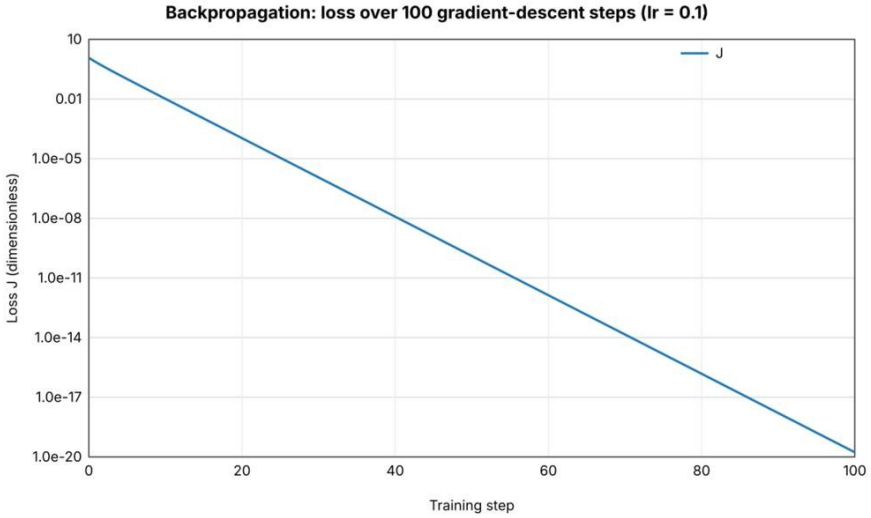


Fig. 3: loss over 100 gradient-descent steps (log scale).

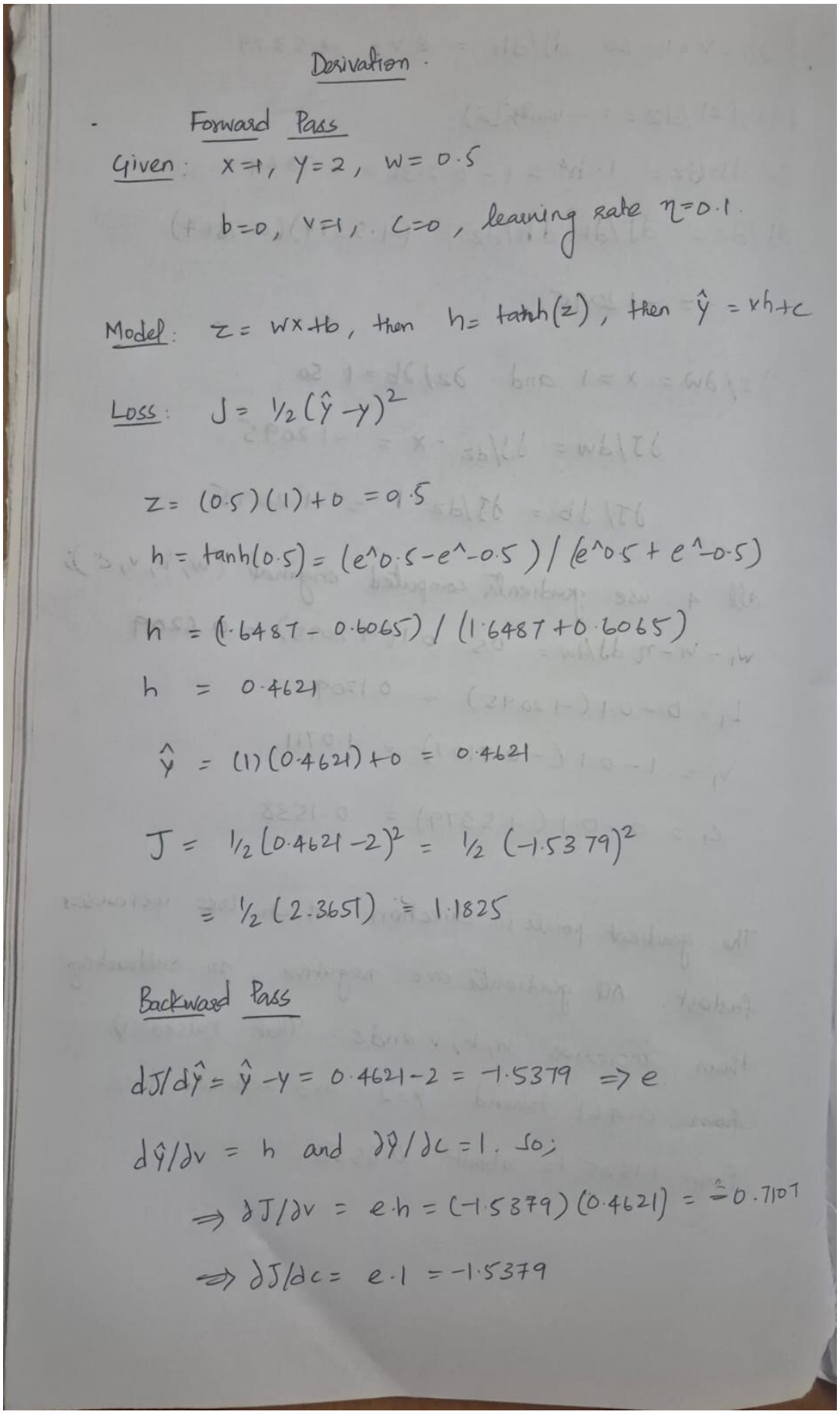
6. Test outcomes

- Five CTest suites (FEA, parsers, batch, backprop, devices) pass on Linux (GCC 13), Windows (MSVC, in CI) and macOS (Apple Clang 17).
- Negative tests: empty, truncated, binary, non-finite and malformed STL/CSV, invalid FE inputs, cancellation, device errors. A ThreadSanitizer run found no data races.
- Packaged Launch.exe was smoke-tested in CI and launched by double-click on a Windows 11 ARM virtual machine (x64 emulation).
- Tested limits: ASCII STL up to 190,000 triangles (45 MB) and CSV up to 1,000,000 rows (40 MB) accepted; larger files are rejected with a message.

7. Limitations

- Parts of the code were written with AI assistance (docs/ai_use_log.md); I reviewed and tested it and can explain it.
- Hardware is simulated; the voltage table is a set of assumptions, not a verified design.
- 1D, constant-property, steady-state thermal model; ASCII STL only; example data are synthetic, not ANSYS exports.
- Launch.exe is unsigned, so SmartScreen may warn. Browser rendering time for the largest files and Windows benchmark timings were not measured.

Appendix: handwritten backpropagation (page 1 of 2)



Derivation

Forward Pass

Given: $x=1,\ y=2,\ w=0.5$

$b=0,\ v=1,\ c=0$, learning rate $\eta=0.1$

Model: $z = wx+b$, then $h=\tanh(z)$, then $\hat{y} = vh+c$

Loss: $J = \frac{1}{2}(\hat{y}-y)^2$

$z = (0.5)(1)+0 = 0.5$

$h = \tanh(0.5) = (e^{0.5}-e^{-0.5})/(e^{0.5}+e^{-0.5})$

$h = (1.6487 - 0.6065)/(1.6487+0.6065)$

$h = 0.4621$

$\hat{y} = (1)(0.4621)+0 = 0.4621$

$J = \frac{1}{2}(0.4621-2)^2 = \frac{1}{2}(-1.5379)^2$

$= \frac{1}{2}(2.3651) = 1.1825$

Backward Pass

$dJ/d\hat{y} = \hat{y}-y = 0.4621-2 = -1.5379 \Rightarrow e$

$d\hat{y}/dv = h$ and $\partial\hat{y}/\partial c = 1$. So;

$\Rightarrow \partial J/\partial v = e\cdot h = (-1.5379)(0.4621) \doteq -0.7107$

$\Rightarrow \partial J/\partial c = e\cdot 1 = -1.5379$

Appendix: handwritten backpropagation (page 2 of 2)

$\partial \hat{y} / \partial h = v = 1$, so $dJ/dh = e \cdot v = +1.5379$

$d \tanh(z)/dz = 1 - \tanh^2(z)$,

So $dh/dz = 1 - h^2 = 1 - 0.2136 = 0.7864$

$dJ/dz = dJ/dh \cdot dh/dz = (+1.5379)(0.7864)$

$= -1.2095$

$\partial z / \partial w = x = 1$ and $\partial z / \partial b = 1$ so.

$\partial J / dw = \partial J / dz \cdot x = -1.2095$

$\partial J / \partial b = \partial J / dz \cdot 1 = -1.2095$

all 4 use gradients computed original (w, b, v, c);

$w_1 = w - \eta \cdot dJ/dw = 0.5 - 0.1(+1.2095) = 0.6209$

$b_1 = 0 - 0.1(+1.2095) = 0.1209$

$v_1 = 1 - 0.1(-0.7107) = 1.0711$

$c_1 = 0 - 0.1(+1.5379) = 0.1538$

The gradient points in direction which loss increases fastest. All gradients are negative, so subtracting them increases w, b, v and c. That raises $\hat{y}$ from 0.461 toward $y = 2$ and reduces loss from 1.1825 to about 0.6858.